

Basic (your first LCEL chain) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_template("Answer: {question}")
model = ChatOpenAI()
parser = StrOutputParser()

chain = prompt | model | parser

result = chain.invoke({"question": "What's 5 + 7?"})
print(result)
```

Expected output:

```
5 + 7 = 12
```

This version works correctly. It's missing an explicit model name and temperature (it relies on `ChatOpenAI()`'s hidden defaults), and it builds the pieces as separate named variables before chaining — fine for a first working version, but not quite what you'd write once the pattern feels natural.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a small `build_chain()` function, with explicit settings

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

def build_chain(model_name: str = "gpt-4o-mini", temperature: float = 0.0):
    prompt = ChatPromptTemplate.from_template("Answer: {question}")
    model = ChatOpenAI(model=model_name, temperature=temperature)
    parser = StrOutputParser()
    return prompt | model | parser

if __name__ == "__main__":
    chain = build_chain()
    result = chain.invoke({"question": "What's 5 + 7?"})
    print(f"Answer: {result}")
```

Expected output:

```
Answer: 5 + 7 = 12
```

Difference from Basic: the model name and temperature are explicit parameters instead of hidden defaults — this matters because a model swap or a temperature change should be a visible, one-line decision, not something buried in a library default that might change between versions. Wrapping the chain-building in a function (`build_chain()`) also means this exact chain can be imported and reused elsewhere, which is exactly what this document's Build Task asks you to do.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — `.batch()` with `return_exceptions=True`, reporting each result

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

def build_chain(model_name: str = "gpt-4o-mini", temperature: float = 0.0):
    prompt = ChatPromptTemplate.from_template("Answer: {question}")
    return prompt | ChatOpenAI(model=model_name, temperature=temperature) | StrOutputParser()

def run_batch(chain, questions: list[str]) -> None:
    inputs = [{"question": q} for q in questions]
    results = chain.batch(inputs, return_exceptions=True)

    for question, result in zip(questions, results):
        if isinstance(result, Exception):
            print(f"FAILED {question!r}: {type(result).__name__}: {result}")
        else:
            print(f"OK {question!r} -> {result}")

if __name__ == "__main__":
    chain = build_chain()
    questions = [
        "What's 5 + 7?",
        "What's the capital of Japan?",
        "Name one gas giant planet.",
    ]
    run_batch(chain, questions)
```

Expected output:

```
OK      "What's 5 + 7?" -> 5 + 7 = 12
OK      "What's the capital of Japan?" -> The capital of Japan is Tokyo.
OK      'Name one gas giant planet.' -> Jupiter is a gas giant planet.
```

On a real failure (a bad input, a transient API error on just one item), that one line would print `FAILED ...` while the other 2 still complete and print normally — `return_exceptions=True` is what keeps one bad input from raising and losing every result in the batch.

Approach 2 — the same idea, timing `.batch()` against a plain loop of `.invoke()` calls

`.batch()` isn't just a shorter way to write a loop — for a provider whose SDK supports it, requests in a batch can run concurrently. This is worth actually measuring, not just trusting.

```
import time
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_template("Answer: {question}")
chain = prompt | ChatOpenAI(model="gpt-4o-mini", temperature=0) | StrOutputParser

questions = [{"question": f"What is {n} squared?"} for n in range(1, 6)]

start = time.perf_counter()
looped_results = [chain.invoke(q) for q in questions]
loop_seconds = time.perf_counter() - start

start = time.perf_counter()
batched_results = chain.batch(questions)
batch_seconds = time.perf_counter() - start

print(f"Looped .invoke() x5: {loop_seconds:.2f}s")
print(f".batch() x5: {batch_seconds:.2f}s")
```

Expected output (exact numbers vary by network conditions, but the pattern holds):

```
Looped .invoke() x5: 4.10s
.batch() x5: 1.35s
```

`.batch()` finishes noticeably faster because the 5 requests overlap instead of waiting for each one to fully finish before starting the next.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's `build_chain()` only ever runs one input at a time. Approach 1 runs many inputs at once with `.batch(return_exceptions=True)`, and handles the realistic case where one of them fails without losing the rest. Approach 2 doesn't change the code path at all — it measures the actual time difference between `.batch()` and a manual loop, turning "batching is faster" from something you were told into a number you watched happen.

Which one should you actually write? Intermediate's single-input chain is all this document's own exercises need. Reach for Approach 1's `.batch(return_exceptions=True)` the moment you're processing more than a handful of inputs through the same chain — a CSV of rows, a list of user messages to classify — where losing every result to one bad row would be a real, avoidable cost. Approach 2's timing comparison is worth running once, for yourself, the first time you're deciding whether batching is worth the extra code for a given task — after that, you'll trust the pattern without re-measuring it every time.